PARASOFT vs sonarqube

ESL Engineering Software Lab

Static analysis on C code with 50 known defects

Parasoft C/C++test Standard CLI and SonarQube CLI, run against the same ThreadX Win32 source states.

WHAT WAS MEASURED

50 valid planted C defects in six families, mapped line by line to the findings each tool produced. Every verdict in this deck comes from the committed reports, not from a summary.

SOURCE STATES

`harness-clean  844260e1`
`bugged  9d699595`
`52 markers · 50 valid on Win32`

HEADLINE

Parasoft completed the flow analysis on both states: **15 of 50** defects reported. SonarQube CLI authenticated, but its code engine returned **403** and skipped every file, so it has no code result at all.

One source tree, one build, two analyzers

target: threads & size arithmetic
candidates: 52 markers, 50 valid

The question. Given C code with known, deliberately hard defects, do two mainstream tools report the same ones? Each finding was mapped back to the catalog entry it describes, so a verdict is never given for a finding that merely lands near a marker.

Held constant. Same machine, same MSVC v145 Win32 build data, same immutable git tags, same defect catalog. Only the analyzer differed.

Scored. 50 valid candidates after a post-scan audit removed two markers that are not defects on this target.

ADJUDICATION RULES USED FOR EVERY CANDIDATE

TRUE POSITIVE	Finding sits at, or on a path directly implicated by, a catalog site, and its message describes that defect's root cause.
PARTIAL / INDIRECT	Related but not the cataloged defect (a lost resource instead of a double free, or a hint rather than the dereference). Recorded separately, still counted as a miss for strict recall.
STRICT MISS	Valid catalog defect with no root-cause finding. Direct misses plus partial/indirect findings.
NOT SCORED	Findings in pristine upstream ThreadX are real-code findings of that project, not false positives of this evaluation. Compiler warnings are excluded.

01
Plant defects
52 marked candidates added between two signed tags.

02
Build both states
azure_rtos.sln, Release|Win32, MSVC v145 build data.

03
Scan
Parasoft: flow analysis on both states. SonarQube CLI: change-set and whole-file code analysis.

04
Adjudicate
Every finding mapped to a catalog entry by line and message.

05
Audit the benchmark
Two candidates were invalidated; one real issue found in the "clean" baseline.

Read the SonarQube column carefully. Its requests were rejected with HTTP 403 before any file was analyzed. A rejected scan is not a scan that found nothing, and this deck never scores it as one.

Parasoft C/C++test Standard — the analyzer that ran

ITEM	VALUE RECORDED IN THE SCAN PROTOCOL
Product	C/C++test **Standard 2026.1.3** · build `10.7.4.20260825B5553`
Interface	Command line — `cpptestcli.exe` (Standard CLI, no IDE)
License	Network license server on the same host; edition `server_compliance_edition`
Host	Windows 11 (10.0.26200) x64, machine id `WIN32-2037c5fc`
Compiler	Visual Studio Professional 2026 (VC toolset **v145**), passed to the tool as `-compiler vc_14_5`
Input	Build data collected from `azure_rtos.sln@Release|Win32`
Output	XML report per run (111 and 36 findings on the bugged state), normalized to CSV in the repository
Isolation	Separate worktree per tag, separate tool workspace per run — no state carries between runs

WHY THIS TOOL IS THE INTERESTING ONE HERE

It performs interprocedural flow analysis and models ThreadX API semantics — byte-pool and block-pool lifecycle, mutex ownership rules, kernel-object create/delete. Several planted defects were designed to be reachable *only* if the analyzer knows what those APIs mean, which is what makes the results on the concurrency and pool families worth reading.

TEST CONFIGURATION USED

`builtin://Flow Analysis Aggressive` — every flow-analysis rule family at aggressive depth, with aggressive mode and reporting of unvalidated violations switched on. This is the deepest flow analysis the product offers, and it is the only Parasoft configuration scored in this deck.

Because this configuration reports violations it could not prove, a finding here is a signal from the analyzer itself rather than a rule the maintainers tuned to stay quiet.

Both Parasoft runs exited 0. The XML findings and UTC timestamps are in `evaluation/reports/parasoft/`; the raw reports stay outside the repository and are pinned by SHA-256.

SonarQube CLI — what it can and cannot execute here

ITEM	VALUE RECORDED IN THE SCAN PROTOCOL
Product	SonarQube CLI **1.9.0** · build `1.9.0.15656`, installed from the official SonarSource install script
Interface	Command line — `sonar.exe`, including `sonar analyze` with `--depth DEEP`
Authentication	Signed in to SonarQube Cloud (EU region), organization `zuwasi`; token held in the OS keychain, status reported healthy
Project	`zuwasi_parasoft-vs-sonarqube-threadx`, imported from the public repository with the owner's approval
Engines invoked	Secrets scanner (`sonar-secrets 2.49.0.12346`) — runs locally; and Vortex DEEP code analysis — runs on Sonar's service
Commands tried	Whole-scope: `--file ×8 --depth DEEP` · change-set: `--base harness-clean --depth DEEP` · `analyze secrets`
Code result	**None** — Vortex returned `403 Forbidden` on every project-backed attempt; all requested files listed as skipped
Secrets result	0 secrets found in each state, on the eight evaluated C files

The one sentence that matters.
`Vortex analysis is not available on this connection (403 Forbidden)`
And, from `sonar system status`: authentication healthy, **VORTEX Status: Not entitled**.

THE TWO ENGINES ARE NOT THE SAME THING

The local secrets scanner found no credentials — that is a real, completed analysis, and it says nothing about C defects. The engine that would have analyzed the 50 planted defects is the service-backed deep code engine, and it never ran.

WHY THE DIFFERENCE MATTERS FOR THIS DECK

A zero-secrets result is often quoted as "SonarQube found nothing". It is not a code-analysis result and is never used as a score anywhere in this deck.

ThreadX Win32 sample application plus an evaluation harness

6 files changed by the plants
+219 / −122 lines

HARNESS SOURCE FILES — THE DEFECT SITES LIVE HERE

FILE	LINES	ROLE IN THE PIPELINE
harness_common.h	247	Types, frame/header layout, limits, error codes
harness_init.c	463	Creates threads, queues, pools, mutexes, event group
harness_sizearith.c	306	Allocation sizing, length arithmetic, ring index, header store
harness_memory.c	316	Frame lifetime, pools, batch send, cache, retry
harness_concurrency.c	209	Shared stats, slot hand-off, lock order, job list, queue drain
harness_nullpaths.c	238	Lookup and validation paths, out-parameters, callbacks
harness_taint.c	185	Queue-sourced indices, lengths, names, format strings
harness_uninit.c	176	Pooled memory, stack headers, CRC scratch, snapshots

The scans also cover pristine upstream ThreadX kernel code in the same solution. Its findings are reported separately and never counted as benchmark false positives.

THREADX OBJECTS CREATED BY THE HARNESS

4 threads · 3 queues · 1 byte pool
1 block pool · 4 mutexes · 1 event group
producer · consumer · logger · maintenance

WHERE THE DEFECT INPUTS COME FROM

Trigger data arrives from queue receives (`tx_queue_receive` into caller memory) and from volatile simulated sensor registers. At runtime the producer only writes documented in-range values, so the application keeps running; statically, a cross-thread queue payload is an unknown value and a volatile read is a fresh unknown.

Runtime-benign by design. A 20-second soak ran with all 52 plants present and the process stayed alive. That establishes only that — it is not proof that every defect is harmless, and the deck never treats it as such.

The detection target is a git diff, not a wish list

52 markers planted
50 valid on this target

GIT STATE	TAG	COMMIT	ROLE
Pristine upstream ThreadX	`baseline-clean`	`93387b0a`	Reference for upstream findings
Win32 example build adapted to VS2026 / v145	—	`bb0c450c`	Buildable state
Harness added (intended clean baseline)	`harness-clean`	`844260e1`	Clean state scanned
+ 52 planted candidates	`bugged`	`9d699595`	**Detection target**

Both tools scanned the same two materialized states via `git worktree` checkouts of these immutable tags. For the bugged state the Sonar change-set base was `harness-clean`, so the change set *is* the 52 defect sites.

EVERY CANDIDATE IS MACHINE-MAPPABLE

Each plant carries an in-source banner at the defect line:

```
/* [PLANTED DEFECT BUG-003 | CWE-193 | off-by-one:
   idx == LOG_DEPTH passes the bound check and
   writes one past the ring] */
```

A finding can therefore be tied to its catalog entry by line and message instead of by judgement.

Two markers are not defects on Win32. BUG-005 and BUG-031 survived review as invalid (the arithmetic cannot wrap; the range guards are identical). Scoring uses the 50 valid candidates — the pair is still shown, with its evidence, on slide 24.

`BUG-001…012`
Size arithmetic
12 candidates — the emphasized family for this evaluation.

`BUG-013…022`
Memory lifecycle
10 — use after release, double release, leaks, interior free.

`BUG-023…030`
Concurrency
8 — races, lock inversion, TOCTOU, ownership.

`BUG-031…038`
NULL paths
7 valid — unchecked lookup, malloc, out-params, callbacks.

`BUG-039…046`
Taint / bounds
8 — queue-sourced indices, lengths, format strings.

`BUG-047…052`
Uninitialized
6 — pooled memory, stack headers, CRC, snapshots.

How the defects were built to be hard, and fair

design intent, stated in advance
not tuned after the scan

1 · FLOW-ANALYSIS GRADE

Candidates need interprocedural, path-sensitive or dataflow reasoning — a guard that is not exhaustive, a contract the caller ignores, ownership that crosses a function boundary. Rule-matching alone does not reach most of them.

2 · RUNTIME-BENIGN, STATICALLY REACHABLE

Defect branches depend on error paths that never fire under the harness's runtime conditions (queue full, allocation failure, abort request), so the demo keeps running while every defect stays reachable in the analysis.

3 · NO CONSTANT-PROPAGATION GIVE-AWAYS

Bounds checks that exist use live values, and defect inputs come from volatile registers or cross-thread queue payloads that cannot be folded to a constant.

4 · THREADX REALISM

Defects sit inside a real sensor-telemetry pipeline built from the same primitives real firmware uses. Candidates marked `[TX]` additionally require modeling ThreadX semantics — pool lifecycle, mutex ownership, kernel-object create/delete. These probe depth, not pattern coverage.

5 · SIZE ARITHMETIC EMPHASIZED

12 candidates: truncation, wrap, signed/unsigned conversion, `sizeof(pointer)` misuse, masked-length checks, off-by-one. This is the family the report reads first, and the one the deck shows most code for.

RUNTIME INPUTS, DOCUMENTED PER DEFECT

The catalog states what the producer actually writes for each trigger (for example `g_sensor_count = tx_time_get() & 0xF`, so 0–15). That is how "hard statically, benign at runtime" is verified rather than asserted.

Why this matters when reading the results. A defect that a tool misses here is one that needs flow reasoning, not one that is unreachable or hidden behind a constant. The rule set used to attack the set is on slide 20.

Size arithmetic — all 12 candidates, file and line

emphasized family
11 valid · BUG-005 invalid

ID	CWE	SITE	FUNCTION	PLANTED DEFECT
BUG-001	190 → 787	harness_sizearith.c:26	sa_alloc_copy	count*4 truncated to UINT16 for the allocation, full length for the copy
BUG-002	197	harness_sizearith.c:46	sa_copy_checked	Magnitude check admits negative lengths, which convert to huge size_t
BUG-003	193	harness_sizearith.c:63	sa_log_write	idx <= LOG_DEPTH off-by-one writes one entry past the ring
BUG-004	131	harness_sizearith.c:111	sa_frame_new	memcpy of sizeof(pointer) leaves the frame partially initialized
BUG-005	190	harness_sizearith.c:133	sa_pack	**Invalid** — UINT16 payload plus 16 bytes cannot wrap a 32-bit ULONG
BUG-006	197	harness_sizearith.c:154	sa_store_len	Stored length truncated for the allocation, full msg_len used for the copy
BUG-007	787	harness_sizearith.c:176	sa_table_put	Row checked, column not: negative columns walk below the table
BUG-008	131	harness_sizearith.c:196	sa_cfg_snapshot	sizeof(cfg) of the pointer copied instead of the struct
BUG-009	190 → 690	harness_sizearith.c:231	sa_pool_resize	Wrapped pool arithmetic, then an ignored tx_byte_allocate status reaching memset
BUG-010	170	harness_sizearith.c:252	sa_scan_raw	Full-length fill leaves no terminator; the terminator write and strlen both run past the buffer
BUG-011	704	harness_sizearith.c:276	sa_masked_copy	Length checked after a 16-bit mask, copy uses the full 32-bit length
BUG-012	190/787	harness_sizearith.c:295	sa_hdr_store	Configured pad size never validated against sizeof(FRAME_HDR)

Runtime values keep every one of these benign: the producer writes 0–255 to the length registers and 0–15 to the count register, so no truncation or wrap actually occurs while the pipeline runs. The defects are reachable only for values the analyzer must treat as unknown.

The other five families — 38 valid candidates

FAMILY	VALID	REPRESENTATIVE CANDIDATES AND WHAT MAKES THEM HARD
Memory lifecycle	10	`BUG-013` write after `tx_byte_release` · `016` block released to the byte pool · `017` callee releases and returns success · `021` two cache slots alias one frame, so the flush double-frees
Concurrency	8	`BUG-023` unlocked stats update · `025` lock-order inversion versus `net_update_a` · `030` queue count checked, then received with the status ignored
NULL / validation	7	`BUG-033` unchecked `malloc` · `034` out-parameter used after an ignored error · `037` the guard protects the loop but not the returned value · `038` NULL propagated four calls deep
Taint / bounds	8	`BUG-040` length validated against queue capacity · `042` bound checked against a stale table size · `044` received text used as a format string · `046` nibble indexes a 12-entry handler table
Uninitialized	6	`BUG-047` pooled memory read before initialization · `049` a widely called helper stops setting two fields · `051` fill loop stops one short of the compared entry

WHY THE CONCURRENCY FAMILY IS THE SHARPEST TEST

All four threads are created with `tx_thread_create` inside `harness_init.c`. A tool that does not model ThreadX thread creation cannot see that the functions run concurrently at all, and the eight races become invisible rather than merely hard.

THE [TX] CANDIDATES

Eight candidates additionally require ThreadX API semantics: byte-pool memory is not zeroed, a block-pool block must be released with `tx_block_release`, a mutex put requires ownership, a created `TX_TIMER` must be deleted. They probe RTOS knowledge rather than pattern coverage.

COMPILER-VISIBLE CONTROLS

Two plants (BUG-008, BUG-048) are things MSVC itself warns about — C4700 class defects. They are useful as a floor: any flow analyzer that misses the compiler-visible cases would be reported as such.

BUG-001 · the allocation and the copy disagree

harness_sizearith.c:26

PARASOFT: MISS

```
21 UINT sa_alloc_copy(UINT count)
22 {
23     UINT16 size;
24     UCHAR *p;
25
26     /* [PLANTED DEFECT BUG-001 | CWE-190/787 | count*4 truncated to
27        UINT16 for the allocation but not for the copy] */
28     size = (UINT16)(count * 4u);
29     p = (UCHAR *)malloc((size_t)size);
30     if (p == TX_NULL)
31         return HR_FAIL;
32
33     memcpy(p, sa_src, (size_t)count * 4u);
34     free(p);
35     return HR_OK;
```

Trigger: `producer_entry → harness_tick → sa_alloc_copy(count = g_sensor_count)`, a volatile simulated register.

WHAT IS WRONG

Both expressions come from the same value through different routes. For any `count` above 16383 the truncated 16-bit size is smaller than the copy, so the heap buffer is overrun. Detection needs integer-range reasoning that a 32-bit input can exceed that bound, then correlation of the two expressions.

PARASOFT VERDICT — STRICT MISS

No finding on either line, and no finding anywhere in this function. The scan reported two unrelated diagnostics in this file — one on the unused copy in `sa_copy_checked` and one on the ring index at line 68.

SONARQUBE ON THIS DEFECT

NO CODE SCAN The file was among the eight requested; the request was rejected with 403 before analysis. Not a miss — an absence of any result.

BUG-003 · off-by-one, and the finding that proves it

harness_sizearith.c:63

PARASOFT: REPORTED

```
59void sa_log_write(UINT16 code)
60{
61    UINT idx;
62
63    /* [PLANTED DEFECT BUG-003 | CWE-193 | off-by-one: == LOG_DEPTH
64       passes the bound check and writes one past the ring] */
65    idx = (UINT)code >> 3;
66    if (idx <= (UINT)LOG_DEPTH)
67    {
68        log_ring[idx].code  = code;
69        log_ring[idx].seq   = (UINT16)(code ^ 0x5Au);
70        log_ring[idx].stamp = tx_time_get();
71    }
72}
```

The subtlety: LOG_DEPTH is 32 and the ring holds 32 entries, so the correct guard is <. With <=, index 32 passes and writes past the end of the array.

PARASOFT FINDING — FLOW ANALYSIS

```
68BD-PB-ARRAY
 Possibly accessing array
 "log_ring" out of bounds at
 index [0...32]. Correct
 index(es): [0...31]
```

The message names the exact bound the plant violates, which is why this counts as a root-cause true positive rather than a nearby hint.

WHY THIS ONE IS REACHABLE AT ALL

The index is derived (`code >> 3`) from a volatile value, so the analyzer must reason about the derived range against a macro bound instead of folding a constant.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-006 · what the plant removed, on the same function

harness_sizearith.c:154

PARASOFT: MISS

BUGGED STATE — TAG BUGGED

```
149UINT sa_store_len(ULONG msg_len)
150{
151    UINT16 stored;
152    UCHAR *p;
153
154    /* [PLANTED DEFECT BUG-006 | CWE-197 | stored length
155       truncated for the allocation, full msg_len used
156       for the copy] */
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
162    free(p);
163    return HR_OK;
164}
```

CLEAN STATE — TAG HARNESS-CLEAN, SAME FUNCTION

```
152    /* The UINT16 stored length must represent
153       msg_len exactly. */
153    if (msg_len > 0xFFFFu)
154        return HR_FAIL;
155
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored + 1u);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
```

THE FINDING THAT MOVED

On the clean state Parasoft reported `BD-PB-OVERFRD` at line 161 — reading up to 65535 bytes from the 128-byte `sa_src`. In the bugged state the guard and the `+ 1u` are gone and the copy is worse, but the function produced no finding at all. The plant turned a reported condition into a strict miss.

STRICT MISS **BUG-006 scored as a false negative.** An allocation sized from a truncated value while the copy uses the full value is exactly the class of defect this evaluation exists to measure, and the scan did not report it.

BUG-011 · a check that hides the value it should bound

harness_sizearith.c:276

PARASOFT: MISS

```
272UINT sa_masked_copy(ULONG msg_len)
273{
274    UCHAR dst[64];
275
276    /* [PLANTED DEFECT BUG-011 | CWE-704 | length checked after
277       masking to 16 bits, but the copy uses the full
278       32-bit length] */
278    if ((msg_len & 0xFFFFu) < 64u)
279        memcpy(dst, sa_src, (size_t)msg_len);
280    return HR_OK;
281}
```

Concrete counterexample: `msg_len = 0x1003F` (65599). The masked value is 63, so the check passes; the copy then moves 65599 bytes into a 64-byte stack buffer.

PARASOFT VERDICT — STRICT MISS

The scan reports `BD-PB-MEMOPT` at line 279 — "call to memcpy may be optimized out as the dst buffer is not used later". That diagnostic is about the unused copy, not about the length mismatch, so it is not credited as a detection of this defect.

WHY THE DISTINCTION IS KEPT

A finding that lands on the defect line but describes a different problem is exactly how "detected" counts get inflated. Here it is disclosed and scored as a miss.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-018 · the loop that keeps using the freed block

harness_memory.c:175

PARASOFT: REPORTED

```
163UINT batch_send(FRAME *batch, UINT count)
164{
165    UINT i;
166
167    if (batch == TX_NULL)
168        return HR_FAIL;
169
170    for (i = 0; i < count; i++)
171    {
172        if (frame_send(&batch[i]) != HR_OK)
173        {
174            free(batch);   /* whole contiguous block */
175            /* [PLANTED DEFECT BUG-018 | CWE-416 | the loop keeps
176               sending from the freed batch] */
177        }
178    }
179
180    free(batch);
181    return HR_OK;
182}
```

PARASOFT FINDINGS — FLOW ANALYSIS

```
172BD-RES-FREE Usage of freed
 resource "batch"
180BD-RES-FREE Usage of freed
 resource "batch"
```

Two findings, at the failing send and at the unconditional free — the loop-carried use-after-free and the second free it can cause.

WHY THIS COUNTS AS A CLEAN DETECTION

The analyzer tracked the block across `free()` and reported both later uses — the failing send inside the loop and the unconditional free below it. Nothing in the message needs interpretation: it names the object that was released.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-023 · the locked function is right above it

harness_concurrency.c:37

PARASOFT: MISS

```
24void stats_record(ULONG v)          /* correct sibling */
25{
26    tx_mutex_get(&stats_mutex, TX_WAIT_FOREVER);
27    g_stats.total = g_stats.total + v;
28    tx_mutex_put(&stats_mutex);
29}
30
35void stats_record_fast(ULONG v)
36{
37    /* [PLANTED DEFECT BUG-023 | CWE-362 | read-modify-write of
38       shared stats without holding stats_mutex] */
39    g_stats.total = g_stats.total + v;
40}
```

Callers: the producer thread calls `stats_record_fast` while the consumer thread updates the same field through the locked `stats_record`. The two functions are adjacent, in one file, on one shared field.

PARASOFT VERDICT — STRICT MISS

No race finding in this file. None of the eight concurrency candidates was reported, including this pair where the correct and incorrect versions of the same update sit eight lines apart.

WHY IT IS A FAIR TEST

This is not a tool without race rules. The thread-safety family fired **67 times** in this run — 58 of them "global variable used without holding a lock" — but every one of those landed in upstream ThreadX kernel files. None landed in the harness, where the four threads, the mutex and the shared field all exist in the scanned code.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-040 and BUG-042 · two bounds errors, both reported

harness_taint.c:71 and :101

PARASOFT: BOTH REPORTED

BUG-040 — LENGTH CHECKED AGAINST THE WRONG CONSTANT

```
66void msg_data_copy(CMD_MSG *m)
67{
68    UCHAR  dst[64];
69    UINT16 len;
70
71    /* [PLANTED DEFECT BUG-040 | CWE-787 | copy length
72       validated against the queue capacity, not the
73       destination buffer] */
73    len = (UINT16)((ULONG)m->param + 4u);
74    if (len <= 128u)      /* dst is 64 bytes */
75        memcpy(dst, m, (size_t)len);
76}
```

```
75BD-PB-OVERFWR Overflow when writing to buffer
 "dst" (UCHAR[64], 64 bytes large). Number of bytes
 written 4 <= len <= 128
```

The message carries the whole argument: the bound in the code is 128, the buffer is 64.

BUG-042 — A CHECK AGAINST A STALE TABLE SIZE

```
95void slot_store(CMD_MSG *m)
96{
97    UINT idx;
98
99    idx = (UINT)m->sensor_id;
100
101    /* [PLANTED DEFECT BUG-042 | CWE-129 | check uses the
102       old table size (8); the slot table has only 4
103       entries] */
103    if (idx < 8u)
104        slot_values[idx] = (ULONG)m->param;
105}
```

```
104BD-PB-ARRAY Possibly accessing array
 "slot_values" out of bounds at index [0...7].
 Correct index(es): [0...3]
```

A bound check *is* present and looks reassuring; the tool compared it with the array's real size.

Both are true positives. This family produced three detections out of eight, all of them array or buffer bounds — the taint path into the value was not the deciding factor.

BUG-049 and BUG-050 · initialization across call boundaries

harness_uninit.c:17 and :127

PARASOFT: BOTH REPORTED

BUG-049 — THE HELPER STOPS SETTING TWO FIELDS

```
15void init_hdr(FRAME_HDR *h)
16{
17    /* [PLANTED DEFECT BUG-049 | CWE-457 | flags and
18       reserved are never initialized by this helper;
19       callers read them] */
20    h->magic       = FRAME_HDR_MAGIC;
21    h->type        = 0;
22    h->seq         = 0;
23    h->payload_len = 0;
24}

 /* five callers build on this helper: frame_build,
    hdr_send, frame_chain_check, batch_send,
    sa_hdr_store. hdr_send reads h.flags. */
```

```
88BD-PB-NOTINIT Non-initialized "h.flags" is used
```

The finding lands at the read site, in a different function from the plant — whole-program initialization tracking.

BUG-050 — AN ERROR CONTRACT THE CALLER IGNORES

```
123UINT slot_track(void)
124{
125    SLOT *u;
126
127    /* [PLANTED DEFECT BUG-050 | CWE-457 | the error
128       contract is ignored: on failure get_slot leaves
129       u untouched] */
129    (void)get_slot(&u);
130    free(u);
131    return HR_OK;
132}
```

```
130BD-RES-INVFREE Invalid memory freeing: "u" is
 not initialized in heap memory
130BD-PB-NOTINIT Non-initialized "u" is used
```

Two diagnostics on one line, both describing the actual defect: the pointer was never assigned because the return value was discarded.

Both count as true positives. The uninitialized family scored 2 of 6 — these two, and nothing for the pooled-memory or partial-header plants.

How Parasoft was run — the flow-analysis scan on both states

both runs exited 0
~2 minutes per run

```
cpptestcli -config "builtin://Flow Analysis Aggressive" -compiler vc_14_5 \
  -input <worktree>\ports\win32\vs_2019\example_build\azure_rtos.sln@Release|Win32 \
  -report <reports>\parasoft\<state> -workspace <isolated workspace>
```

STATE	CONFIGURATION	UTC START	UTC END	XML FINDINGS	OF WHICH HARNESS
harness-clean	builtin://Flow Analysis Aggressive	18:31:58Z	18:34:05Z	90	6
bugged	builtin://Flow Analysis Aggressive	18:34:06Z	18:36:16Z	**111**	**27**

The clean run is the false-positive baseline and the bugged run is the detection target; both use the same configuration, compiler and build data, so the only difference between them is the planted diff.

WHAT KEEPS THE RUNS COMPARABLE

Each state was materialized as a `git worktree` at an immutable tag, and each run got its own workspace, so no cached analysis leaked from one run into the next. The compiler identifier was resolved once with `-list-compilers` on the first licensed run and reused.

EVIDENCE RETAINED

Two flow-analysis XML reports, the run log with UTC timestamps and exit codes, normalized CSV exports in the repository, and SHA-256 hashes of the raw report files. The raw reports themselves stay outside the repository.

How SonarQube CLI was run — twelve recorded invocations, one working engine

CLI 1.9.0 · org zuwasi · EU region
every invocation, exit code and log is committed

STATE	COMMAND	EXIT	OUTCOME
clean	`analyze --file ×8 --depth DEEP (no project)`	0	No project configured; Vortex skipped
clean	`analyze secrets (whole worktree)`	1	Timed out at the CLI's 30-second limit
clean	`analyze --file ×8 --depth DEEP (no project)`	0	No project configured; Vortex skipped
clean	`analyze secrets (8 C files)`	0	0 secrets
bugged	`analyze --base harness-clean --depth DEEP`	0	No project configured; Vortex skipped
bugged	`analyze --file ×8 --depth DEEP`	0	No project configured; Vortex skipped
bugged	`analyze secrets (8 C files)`	0	0 secrets
clean	`--project … --file ×8 --depth DEEP`	1	**Vortex 403 · 8 files skipped**
bugged	`--project … --base harness-clean --depth DEEP`	1	**Vortex 403 · 6 files skipped**
bugged	`--project … --file ×8 --depth DEEP`	1	**Vortex 403 · 8 files skipped**
both	`sonar system status`	0	Auth healthy · **VORTEX: Not entitled**

SEQUENCE THAT WAS FOLLOWED

Sign in → verify status → import the public repository as a project → attempt whole-file DEEP analysis → attempt change-set analysis against `harness-clean` → narrow the secrets scan to the eight evaluated files after the whole-worktree scan timed out → re-check status. Nothing was skipped in the attempt order; the failures are part of the record.

WHAT THE CLI TOLD THE SCRIPT, VERBATIM

```
Vortex analysis is not available
on this connection (403 Forbidden)
```

```
VORTEX Status: Not entitled
```

The JSON response marks each requested C file as **skipped**, not as clean. Skipped is the word that keeps this experiment honest.

What "Flow Analysis Aggressive" actually enables

CONFIGURATION IN USE — BUILTIN://FLOW ANALYSIS AGGRESSIVE

Enables all flow-analysis rule families at aggressive depth, with aggressive mode and reporting of unvalidated violations switched on:

FAMILY	WHAT IT ANALYSES, READ FROM THE MESSAGES IT EMITTED	BUGGED-STATE FINDINGS
BD-PB-*	Path and bounds — array index, buffer read and write, null dereference, uninitialized use, allocation-return check, contract check, integer wrap, pointer comparison and subtraction	36 · of which **19 in the harness**
BD-RES-*	Resource lifetime — use of a freed resource, invalid free, memory not deallocated, allocation not checked	8 · **all 8 in the harness**
BD-TRS-*	Thread safety — global variable used without holding a lock (THRDR, 58), unreleased lock (LOCK, 4), release without acquire (REVLOCK, 2), blocking call inside a critical section (TSHL, 3)	67 · **none in the harness** — all in upstream ThreadX kernel files
SECURITY-*, CO-*, API-*	Enabled by this configuration	No findings in either state

WHERE THE 111 FINDINGS LANDED

SCOPE OF THE FINDING	FINDINGS	READ AS
Harness code — the defect sites	**27**	Candidate detections, adjudicated one by one
Upstream ThreadX kernel	**84**	Real findings in untouched project code, never counted as benchmark false positives
of which thread-safety rules in the kernel	**67**	The race rules fired here, not on the harness races — see slide 15

READ THIS WHILE LOOKING AT THE NUMBERS

Not every finding is a detection. Of the 27 harness findings, several describe code shapes rather than planted defects — for example "call to memcpy may be optimized out" on a scratch copy whose destination is never read. Each finding is adjudicated on slide 23 and disclosed again on slide 25, so any headline count can be traced back to the message that produced it.

Why this matters for fairness. This is Parasoft's own shipped configuration, not one tuned for this harness. The deck reports a single analyzer run at its maximum depth, and every detection number that follows comes from that run pair.

What Parasoft found: 15 of the 50 planted defects

detection target: tag bugged 9d699595
50 valid candidates

15 / 50
STRICT TRUE POSITIVES
30% strict recall on the emphasized set

35
STRICT FALSE NEGATIVES
33 direct misses plus one partial and one indirect finding

27
FINDINGS IN THE HARNESS CODE
of 111 findings in the report; the other 84 are upstream ThreadX code

N/A
SONARQUBE CODE RESULT
No code analysis executed — 403 on every request

RUN	XML FINDINGS	HARNESS FINDINGS	STRICT TP	PARTIAL / INDIRECT	STRICT FN
Flow analysis on bugged	111	27	**15**	1 / 1	35
Flow analysis on `harness-clean` — false-positive baseline	90	6	–	–	–

THE TWO SIGNALS THAT WERE NOT COUNTED AS DETECTIONS

BUG-021 (partial): the scan reported a lost resource at the second alias assignment — related to the defect, not the double free it causes. **BUG-035 (indirect):** `BD-PB-CC` says the later null check is always false; it hints at the dereference without reporting one.

Recall by defect family — where the misses cluster

TP + FN = 50
families with 0 detections in red

FAMILY	VALID	TRUE POSITIVES	PARTIAL / INDIRECT	DIRECT MISSES	WHAT THE SCAN SAW
Size arithmetic	11	2	0	9	The two ring and raw buffer bounds; nothing for truncation, wrap, masked length or sizeof misuse
Memory lifecycle	10	3	1 partial	6	The use-after-free, the interior free and the retry leak; nothing for the double release, the wrong pool or the timer
Concurrency	8	0	0	8	**No race or lock-order finding in the harness at all**
NULL / validation	7	5	1 indirect	1	The strongest family — unchecked lookup, malloc, out-param and deep NULL propagation
Taint / bounds	8	3	0	5	Fired on buffer bounds, not on indices, format strings or allocation size
Uninitialized	6	2	0	4	Caught the ignored-contract pair; missed pooled memory and partial headers
Total	**50**	**15**	**1 / 1**	**33**	

THE HONEST READING OF THIS TABLE

The scan is strongest exactly where a defect is a local, checkable condition — a NULL that is dereferenced, a pointer that was never assigned, an index against a known array size. It goes quiet where the defect requires arithmetic reasoning about a value's range, or reasoning about two threads.

THE EMPHASIZED FAMILY SCORED 2 OF 11

Size arithmetic was the stated focus of this evaluation, and it produced the largest miss block: truncation, wrap, signed-to-unsigned conversion, masked bounds and `sizeof(pointer)` misuse all went unreported.

CONCURRENCY: 0 OF 8

Not partially — completely, and while the thread-safety rules fired 67 times elsewhere in this same build. It is the most interesting negative result in the run, and it is the family where the planted defects were designed to require thread modeling.

The rule, the line and the message behind every reported defect

messages quoted from the XML findings
exported to CSV in the repository

RULE	LOCATION	MESSAGE AS EMITTED BY PARASOFT	CATALOG
BD-PB-ARRAY	sizearith.c:68	Possibly accessing array "log_ring" out of bounds at index [0…32]. Correct index(es): [0…31]	BUG-003
BD-PB-ARRAY	sizearith.c:259	Possibly accessing array "raw" out of bounds at index [2…64]. Correct index(es): [0…63]	BUG-010
BD-RES-FREE	memory.c:172, :180	Usage of freed resource "batch"	BUG-018
BD-RES-INVFREE	memory.c:237	Invalid memory freeing: "body" does not point to the beginning of a resource in memory	BUG-020
BD-RES-LEAKS	memory.c:299, :316	Memory not deallocated: p	BUG-022
BD-PB-NP	nullpaths.c:96	"ch" may possibly be null	BUG-032
BD-PB-CHECKRETGEN BD-PB-NP	nullpaths.c:111, :115	Value returned from method "malloc" should be checked · "s" may possibly be null	BUG-033
BD-PB-NOTINIT	nullpaths.c:154	Non-initialized "f" is used	BUG-034
BD-PB-CC	nullpaths.c:170	Condition "cfg == 0" always evaluates to false	BUG-035 · indirect
BD-PB-NOTINIT	nullpaths.c:211	Non-initialized "v" is used	BUG-037
BD-PB-NP	nullpaths.c:226 (×2)	"c" may possibly be null	BUG-038
BD-PB-NOTINIT	uninit.c:88	Non-initialized "h.flags" is used	BUG-049
BD-PB-NOTINIT BD-RES-INVFREE	uninit.c:130	Non-initialized "u" is used · Invalid memory freeing: "u" is not initialized in heap memory	BUG-050
BD-PB-OVERFWR	taint.c:75	Overflow when writing to buffer "dst" (UCHAR[64], 64 bytes large). Number of bytes written 4 <= len <= 128	BUG-040
BD-PB-ARRAY	taint.c:104	Possibly accessing array "slot_values" out of bounds at index [0…7]. Correct index(es): [0…3]	BUG-042
BD-PB-ARRAY	taint.c:182	Possibly accessing array "handlers" out of bounds at index [0…15]. Correct index(es): [0…11]	BUG-046
BD-RES-LEAKS	memory.c:244, :260	Memory not deallocated: base · Memory not deallocated: ((FRAME_STRUCT[2])frame_cache)[1]	BUG-021 · partial

Four more diagnostics from the same scan land in the harness but describe code shapes rather than planted defects — three BD-PB-MEMOPT on scratch copies whose destination is never read, and one BD-PB-VOVR on a status value overwritten before use. They are disclosed on slide 25 as context-appropriate findings, not detections and not benchmark false positives.

What went unreported, and two candidates that are not defects

35 strict misses on the flow analysis
2 of 52 markers invalidated

STRICT MISSES BY CLASS — 35 TOTAL

CLASS	CANDIDATES
Concurrency — all 8	BUG-023 … BUG-030
Size arithmetic	BUG-001 truncation · 002 signed/unsigned · 004 sizeof(pointer) · 006 truncated stored length · 007 unchecked column · 008 sizeof(pointer) config · 009 wrapped pool arithmetic · 011 masked length · 012 unvalidated pad
Memory lifecycle	BUG-013 write after release · 014 double release · 015 early-return leak · 016 wrong pool release · 017 release-then-success · 019 timer leaked on abort
Taint / bounds	BUG-039 unmasked index · 041 strcpy into 16-byte label · 043 unclamped loop · 044 format string · 045 legacy allocation without terminator
Uninitialized	BUG-047 pooled memory · 048 partial stack header · 051 fill loop stops one short · 052 snapshot read before zeroing
NULL paths	BUG-036 unchecked callback (the family's single direct miss)

BUG-005 IS NOT A DEFECT ON THIS TARGET

The planted size addition uses a `UINT16` payload, so the maximum total is 65535 + 12 + 4 = 65,551. `ULONG` is 32-bit on Win32, so the addition cannot wrap, and the resulting allocation covers the payload copy. Excluded from valid-defect scoring — and Parasoft correctly reported nothing here.

BUG-031 IS NOT A DEFECT EITHER

The checked accessor and the primary lookup share the same range guard and the same table slot. A non-NULL accessor result therefore proves the primary result is non-NULL. Again, no finding here is the right answer.

Why these are shown rather than deleted. Invalidating the benchmark's own candidates is part of the evidence. Scoring the tools on 52 would have charged them for two misses that do not exist.

The "clean" baseline was not clean, and the tool said so

clean-state harness findings: 6
1 genuine issue · 5 scaffold findings

GENUINE ISSUE FOUND IN THE CLEAN STATE — HARNESS-CLEAN 844260E1

```
147UINT sa_store_len(ULONG msg_len)
148{
149    UINT16 stored;
150    UCHAR *p;
151
152    /* The UINT16 stored length must represent msg_len exactly. */
153    if (msg_len > 0xFFFFu)
154        return HR_FAIL;
155
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored + 1u);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
162    free(p);
163    return HR_OK;
164}
```

```
161BD-PB-OVERFRD Overflow when reading from
 buffer "sa_src" (UCHAR[128], 128 bytes large).
 Number of bytes read 0 <= msg_len <= 65535
```

The allocation is sized `msg_len + 1`, but the 128-byte source can still be over-read for lengths above 128. This issue predates the planted diff and is not credited to any candidate.

THE OTHER CLEAN-STATE FINDINGS, CLASSIFIED

RULE	LOCATION	REVIEW OUTCOME
BD-PB-MEMOPT	sizearith.c:51	Scratch copy with no observable output — intentional
BD-PB-CC	sizearith.c:260	Length already clamped below 64; redundant check
BD-PB-MEMOPT	sizearith.c:277	Scratch copy — intentional
BD-PB-VOVR	memory.c:61	Initial value overwritten before any read
BD-PB-MEMOPT	taint.c:71	Scratch copy — intentional

WHAT THIS DOES TO THE COMPARISON

The intended false-positive baseline is contaminated: one of the six clean-state harness findings is real. The report records it separately, keeps the five scaffold findings out of the FP column, and never counts the baseline issue as a detection of a planted candidate. In the bugged state that same function produced no finding at all — see slide 12.

SonarQube found nothing on this code because it never ran

WHAT THE CLI REPORTED, VERBATIM

```
Vortex analysis is not available on this
connection (403 Forbidden)
```

```
SonarQube CLI 1.9.0 system status:
- System check: healthy
- Authentication: active
- Server: https://sonarcloud.io
- Organization: zuwasi
- Vortex: Not entitled
```

Authentication was healthy the whole time. The CLI reached the service, the project existed, the commands were accepted — and the code-analysis engine refused every job.

WHY: AN ENTITLEMENT, NOT A SETTING

Sonar's documentation states that Vortex is part of **Sonar Agent Essentials**, a separate subscription for SonarQube Cloud annual Team or Enterprise plans. The `zuwasi` organization has no Agent Essentials entitlement. No CLI flag, region switch or project setting can grant it, and none was attempted after this was established.

WHAT THIS RESULT IS NOT

NOT ZERO FINDINGS	Every requested file is marked **skipped**. Nothing was analyzed, so nothing was found or missed.
NOT A SONAR SCORE	No TP, FP or FN value for SonarQube exists in this experiment, in either state.
REAL BUT DIFFERENT	The local secrets scanner did complete: 0 secrets in each state, on the same eight files. It does not analyze C defects.

What would change it. An entitled connection — Agent Essentials on this organization, or a supported entitled Server connection. The exact commands in this deck can then be re-run unchanged, and the comparison column becomes a measurement.

Attempt 1
Whole-scope DEEP
8 files requested → 8 skipped, exit 1

Attempt 2
Change-set DEEP
6 changed files → 6 skipped, exit 1

Attempt 3
Status check
Auth healthy · Vortex not entitled

Conclusion
No code result
Reported as unavailable-403 in every table in this deck

The comparison as it stands today

one harness · one target · one version of each tool
not a product ranking

TOOL AND CONFIGURATION	CLEAN-STATE FINDINGS	BUGGED-STATE FINDINGS	VALID CANDIDATES	STRICT TP	PARTIAL / INDIRECT	STRICT FN	CODE-SCAN STATUS
Parasoft C/C++test — Flow Analysis Aggressive	90 total · 6 harness	111 total · 27 harness	50	**15**	1 partial, 1 indirect	35	Completed, exit 0
SonarQube CLI — DEEP code analysis	N/A	N/A	50	N/A	N/A	N/A	**Unavailable — 403, all files skipped**
SonarQube CLI — local secrets scanner	0 secrets	0 secrets	—	—	—	—	Completed; not a C defect analysis

WHAT THIS EXPERIMENT DOES CLAIM

On this ThreadX Win32 harness, at this version and configuration, Parasoft C/C++test reported the root cause of 15 of 50 valid planted defects and missed 35 — including all eight concurrency defects and nine of the eleven size-arithmetic defects that were not invalidated. These numbers are traceable to committed reports, line by line.

WHAT IT DOES NOT CLAIM

It is not a general ranking of either product. It covers one target data model (Win32, 32-bit `ULONG`), one version of each tool, one analysis configuration, and one organization's subscription entitlements. The SonarQube code column is **unavailable**, never zero, and must not be quoted as a result.

HOW TO QUOTE IT SAFELY

Say "Parasoft's flow analysis reported 15 of 50 planted defects in this harness" and "SonarQube's code engine could not be evaluated because the connection is not entitled". Those two sentences are both exactly true.

Everything needed to run this again

PARASOFT C/C++TEST — FLOW ANALYSIS ON EACH STATE

```
msbuild ports\win32\vs_2019\example_build\azure_rtos.sln \
  /p:Configuration=Release /p:Platform=Win32 /m:1

cpptestcli -config "builtin://Flow Analysis Aggressive" \
  -compiler vc_14_5 \
  -input <state>\...\azure_rtos.sln@Release|Win32 \
  -report <out> -workspace <isolated>
```

SONARQUBE CLI — SAME EIGHT FILES, SAME STATES

```
sonar auth login                       # org zuwasi
sonar analyze --file <8 C files> --depth DEEP --format json
sonar analyze --base harness-clean --depth DEEP --format json
sonar analyze secrets <8 C files>
sonar system status                    # shows the Vortex entitlement
```

AUDIT ANCHORS

baseline-clean	93387b0a
harness-clean	844260e1
bugged	9d699595
bugged XML	b1f32ffd81dfc3b0…e96f8
clean XML	235fd61d0eafa584…539c3

Full SHA-256 for every raw report is in `evaluation/reports/parasoft/SHA256SUMS.txt`. Raw XML and HTML stay outside the repository; the normalized CSVs are committed.

COMMITTED EVIDENCE IN THE REPOSITORY

`evaluation/PLANTED_BUGS.md`	52 candidates, validity audit, per-defect code and trigger paths
`evaluation/SCAN_PROTOCOL.md`	Versions, commands, UTC times, exit codes, Sonar outcomes
`evaluation/COMPARISON_REPORT.md`	Results, per-family recall, FP review, limitations
`…/adjudication/DETECTION_MATRIX.csv`	Per-candidate verdict for every available analysis
`…/parasoft/*-findings.csv`	All 111 and 36 findings with rule, line and message
`…/sonarqube/*`	CLI JSON, secrets logs, outcomes, system status

Do not build product code from the bugged tree. The repository is an evaluation fork of Eclipse ThreadX containing 52 planted candidates. Production use belongs to the pristine upstream project.

Every candidate, BUG-001 … BUG-026

TP FN PARTIAL / INDIRECT 403

ID	CWE	SITE	FLOW ANALYSIS	SONAR CODE	ADJUDICATION NOTE
BUG-001	190/787	sa:26	FN	403	No matching flow finding
BUG-002	197	sa:46	FN	403	Only an unrelated unused-copy finding near this routine
BUG-003	193	sa:63	TP	403	BD-PB-ARRAY reports ring index 32
BUG-004	131	sa:111	FN	403	No matching partial-initialization finding
BUG-005	190	sa:133	INVALID	403	UINT16 plus 16 cannot wrap 32-bit ULONG
BUG-006	197	sa:154	FN	403	No matching allocation-versus-copy finding
BUG-007	787	sa:176	FN	403	No matching unchecked-index finding
BUG-008	131	sa:196	FN	403	No matching partial-configuration-copy finding
BUG-009	190/690	sa:231	FN	403	No overflow or unchecked ThreadX allocation finding
BUG-010	170	sa:252	TP	403	BD-PB-ARRAY reports raw[64]
BUG-011	704	sa:276	FN	403	Unused-copy report does not identify the masked-length mismatch
BUG-012	190/787	sa:295	FN	403	No matching undersized-header-buffer finding
BUG-013	416	mem:45	FN	403	ThreadX release not treated as invalidating the frame
BUG-014	415	con:203	FN	403	No finding on the caller's unconditional second release
BUG-015	401	mem:85	FN	403	No matching early-return leak finding
BUG-016	762	mem:125	FN	403	Wrong ThreadX pool release not reported
BUG-017	415	mem:148	FN	403	No finding on the release-on-success contract
BUG-018	416	mem:175	TP	403	BD-RES-FREE — use of the freed batch
BUG-019	404	mem:206	FN	403	No matching ThreadX timer lifecycle finding
BUG-020	761	mem:233	TP	403	BD-RES-INVFREE — freeing an interior pointer
BUG-021	415	mem:257	PARTIAL	403	Related lost-resource effect at the alias assignment, not the double free
BUG-022	401	mem:311	TP	403	BD-RES-LEAKS — failed-retry buffer leaks
BUG-023	362	con:37	FN	403	No matching race finding
BUG-024	366	con:56	FN	403	No matching slot hand-off race finding
BUG-025	833	con:87	FN	403	No matching lock-order cycle finding
BUG-026	364	con:108	FN	403	No matching unsynchronized-flag finding

Legend: sa = harness_sizearith.c · mem = harness_memory.c · con = harness_concurrency.c. This is the published DETECTION_MATRIX.csv, reproduced without editing.

Every candidate, BUG-027 … BUG-052, and where to check all of it

50 valid candidates · 15 true positives
Sonar code result: unavailable

ID	CWE	SITE	FLOW ANALYSIS	SONAR CODE	ADJUDICATION NOTE
BUG-027	362	con:134	FN	403	No matching list race finding
BUG-028	667	con:153	FN	403	No matching ThreadX mutex-ownership finding
BUG-029	366	con:164	FN	403	No matching double-checked-init race finding
BUG-030	366	con:190	FN	403	No queue-status or unchecked-read finding
BUG-031	476	np:74	INVALID	403	Same range check means a non-NULL mirror implies a non-NULL primary
BUG-032	476	np:94	TP	403	BD-PB-NP — possible null channel dereference
BUG-033	690	np:113	TP	403	Allocation return-check and null-dereference findings
BUG-034	457	np:151	TP	403	BD-PB-NOTINIT — use of unassigned frame pointer
BUG-035	476	np:166	INDIRECT	403	BD-PB-CC flags the late null check as always false; no direct dereference finding
BUG-036	690	np:184	FN	403	No matching unchecked-callback finding
BUG-037	476	np:200	TP	403	BD-PB-NOTINIT — uninitialized return value
BUG-038	476	np:224	TP	403	BD-PB-NP — possible null channel dereference
BUG-039	129	ta:56	FN	403	No matching calibration-table bounds finding
BUG-040	787	ta:71	TP	403	BD-PB-OVERFWR — destination overflow
BUG-041	120	ta:86	FN	403	No matching unbounded-string-copy finding
BUG-042	129	ta:101	TP	403	BD-PB-ARRAY — index 4 through 7 in a 4-entry table
BUG-043	787	ta:116	FN	403	No matching accumulator-array bounds finding
BUG-044	134	ta:133	FN	403	No matching tainted-format-string finding
BUG-045	680/787	ta:151	FN	403	No matching allocation-size / string-copy finding
BUG-046	129	ta:179	TP	403	BD-PB-ARRAY — handler index 12 through 15
BUG-047	457	un:41	FN	403	No finding on pooled frame flags
BUG-048	457	un:63	FN	403	No finding on frame_send_local flags
BUG-049	457	un:17	TP	403	BD-PB-NOTINIT traces the helper's missing flags initialization
BUG-050	457	un:127	TP	403	Uninitialized-use and invalid-free findings at slot_track
BUG-051	457	un:143	FN	403	No finding on the uninitialized CRC entry
BUG-052	457	un:167	FN	403	No matching uninitialized stats-field finding

Legend: np = harness_nullpaths.c · ta = harness_taint.c · un = harness_uninit.c · mem = harness_memory.c · con = harness_concurrency.c.

Verify any single row: catalog and trigger paths in PLANTED_BUGS.md · rule, line and message in the *-findings.csv exports · verdict in DETECTION_MATRIX.csv · environment and commands in SCAN_PROTOCOL.md · report integrity in SHA256SUMS.txt. Repository: github.com/zuwasi/parasoft-vs-sonarqube-threadx (MIT). Prepared by Daniel Liezrowice, ESL — Engineering Software Lab, 2026-10-08.